

Algoritmusok és adatszerkezetek I.

Hasító Táblák

Bevezetés

- ◊ A hasító táblák (hash tables) célja egy olyan adattárolási eljárás megvalósítása, amelyben a **keresés, beszúrás és törlés** (szótár)-műveletek várhatóan nagyon hatékonyak.
- ◊ (Akár elsőre megtalálható minden keresett kulcs.)
- ◊ Feltesszük, hogy a tárolandó rekordoknak van egy egyedi kulcs adattagja.
- ◊ A kulcsok egy U halmaz elemei (U a kulcsok univerzuma), amiről ezúttal nem szükséges feltételezni, hogy rendezhető.

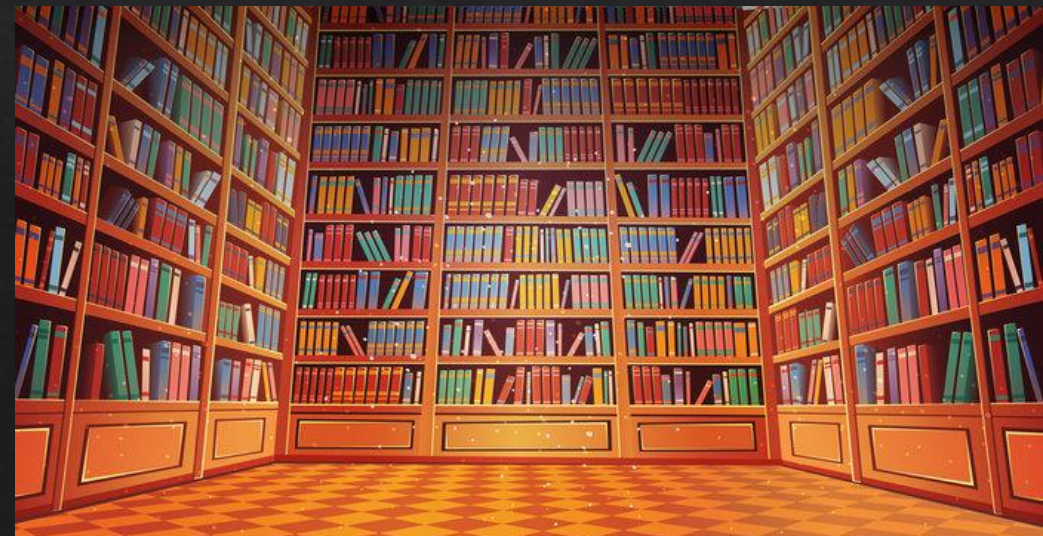
Hasítás, hasító táblák

- ◆ Céljuk: **hatékony keresés**, viszonylag állandónak tekinthető kulcskészleten (kulcsok egyediek), de legyen lehetőség módosításra (beszúrás, törlés).
- ◆ Nézzük, amink van:
 1. Rendezetlen tömb (lista) + lineáris keresés $O(n)$
 2. Rendezett tömb + logaritmikus keresés $O(\log n)$ (módosítások: $O(n)$)
 3. Rendezett lista + hatékonyabb lineáris keresés $O(n)$ (módosítások: $O(n)$)
 4. Bináris keresőfa: keresés, módosítások a fa magasságával arányosak, ami $\log n..n$ intervallumban lehet.
 5. Hasítás: keresés jó hatékonyságú hasító tábla esetén konstans lépésszámú (akár 1-2 lépés) (módosító műveletek is hasonlóan hatékonyak).

Illusztrációs probléma

- ◊ Tegyük fel, hogy szeretnél feltölteni egy könyvtárat könyvekkel, de nem csak úgy bedobálni őket, hanem később is könnyedén szeretnéd megtalálni őket, amikor szükséged van rájuk.
- ◊ A könyv címe alapján azonnal meg tudjuk mondani a megfelelő helyet, így csak oda kell sétálnod a megfelelő polchoz, és leemelned a könyvet.
- ◊ De hogyan lehet ezt megvalósítani?
- ◊ Nos, egy kis előrelátással – ahelyett, hogy csak sorban kezdenéd el megtölteni a könyvtárat – kidolgozol egy okos kis módszert: fogod a könyv címét, lefuttatod egy kis számítógépes programon, amely kiad egy polcszámot és egy helyszámot azon a polcon. Ide teszed a könyvet.
- ◊ Ennek a programnak a szépsége, hogy amikor valaki később visszajön, hogy elolvassa a könyvet, újra lefuttatjuk a címet a programon, és ugyanazt a polcszámot és helyszámot kapjuk vissza, amit eredetileg is kaptunk – ott található a könyv.

Source: <https://stackoverflow.com/a/730813>



Source: https://stock.adobe.com/search?k=library+cartoon&asset_id=266015225

I. Direkt címzés

Direkt címzés - Bevezetés

- ◊ A példa kedvéért vegyünk egy táblázatot, melyben a 8, 3, 13, 6, 4, 10-es kulcsokat (értékeket) szeretnénk eltárolni.

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

- ◊ A létező legegyszerűbb és kézenfekvőbb megoldás, ha a kulcsokat az értékükkel megegyező indexen helyezzük el, hiszen így az értékeket eleve jó sorrendbe tároltuk el.

			3	4		6		8		10			13	
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

Az így eltárolt adatokat könnyen megtalálhatjuk a tömbünkben, és ha a kulcs benne van a táblában, akkor visszakapjuk annak értékét, egyébként NIL. Az ilyen módon eltárolt elemek elérése (keresése) $O(1)$.

- ◊ Azonban egy problémával szembesülünk: mi történik akkor, ha egy új elem értéke nagyobb a tömb méreténél?

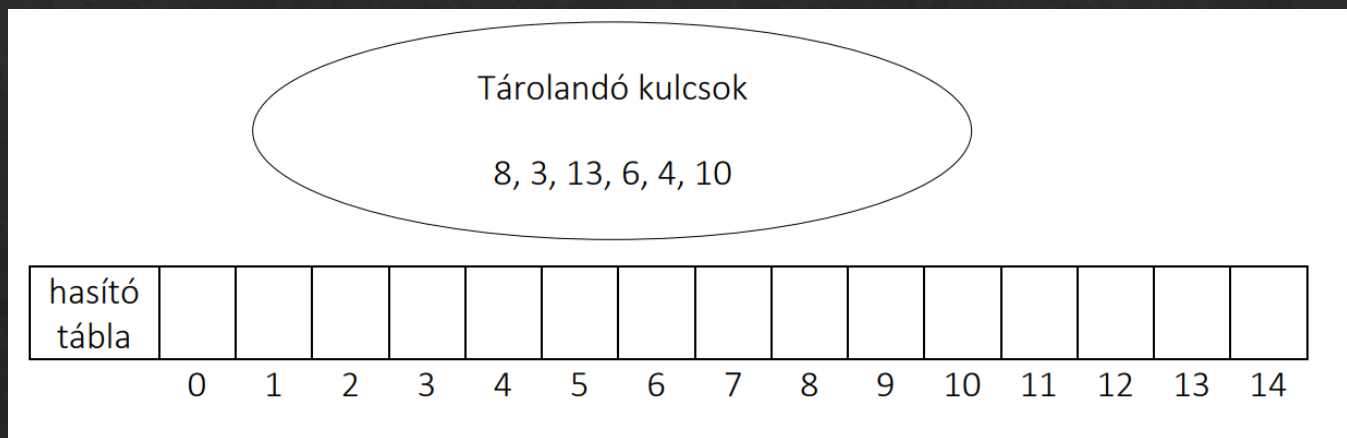
Kulcsok: 8, 3, 13, 6, 4, 10, 50

			3	4		6		8		10			13	
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

?

Direkt címzés - Bevezetés

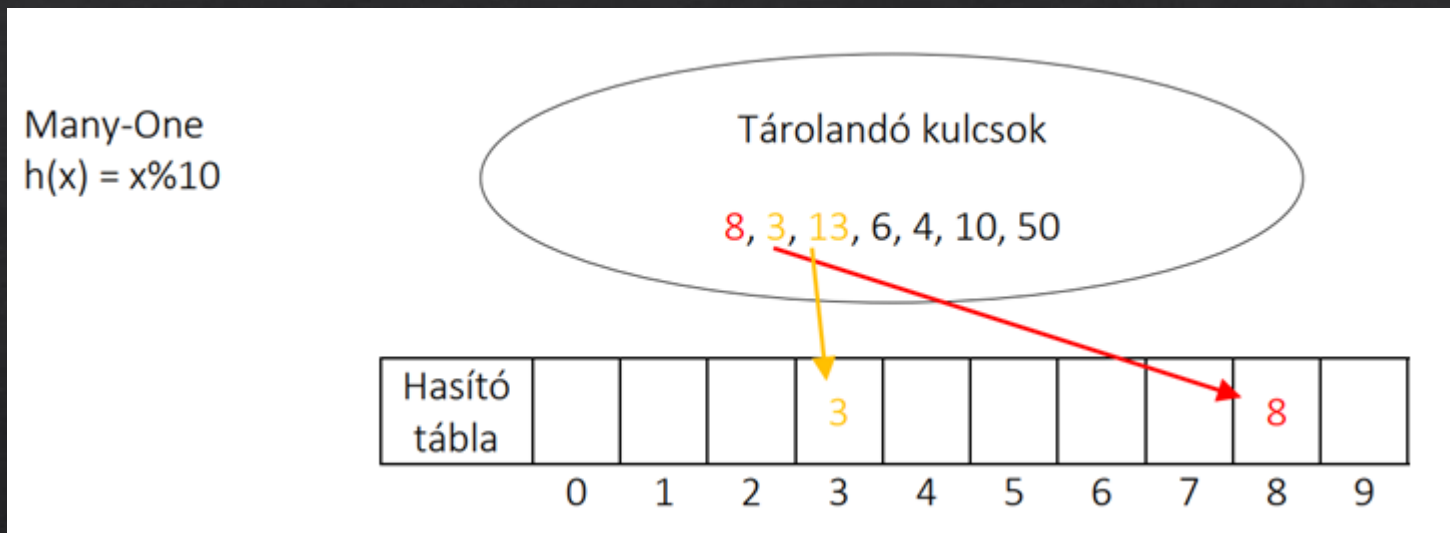
- ◊ Egy megoldás lenne, ha megnövelnénk a tömb méretét, viszont emiatt sok felesleges helyet foglalnánk le. A probléma megoldására a HASÍTÓ függvények gondolatát vezettük be. Nézzünk egy példát a hasító függvények alkalmazására.
- ◊ Egy nagyon „primitív megoldásban” a már előzőekben megismert, ÉRTÉK == KULCS/INDEX függvény is alkalmazható lenne.



- ◊ Viszont, ha egy új 50-es értéket adunk a halmazunkhoz, akkor azzal nem tudunk mit kezdeni. Mielőtt továbbmennénk, le kell tisztáznunk, hogy van ONE-ONE és MANY-ONE típusú függvény. Az imént látott $h(x) = x$ hasító függvény egy ONE-ONE megoldás. Ahhoz, hogy megoldjuk a problémát, át kell írunk a módosító függvényt ($h(x) = x$) valami jobb megoldásra.

Direkt címzés - Bevezetés

- ❖ Módosítsuk úgy a feladat megoldását, hogy a $h(x) = x$ -et cseréljük fel a $h(x) = x \% 10$ formulára, ahol a 10 a tárolótömb méretét hivatott ábrázolni. Teszteljük a megoldást. Néhány szám erejéig jól működik az elgondolás. A 8-as és a 3-as érték bekerülhet a tömbbe, ám a 13 modulusa 3, így ütközés áll fenn, mivel nem kerülhet két elem ugyanarra a helyre.



- ❖ Mi ilyenkor a teendő? A hasító táblák alapja a „megfelelően” megválasztott hasító függvény. A rosszul megválasztott függvény kulcsütközéshez (collision) vezet. Gyakori hiba továbbá, a kulcsok csoportosulása (clustering). Ebben az esetben annak a valószínűsége, hogy több kulcs ugyanazt az indexet adja, jóval nagyobb, mintha egy véletlenszám-generátor függvényt használtunk volna.

Direkt címzés - Bevezetés

- ◊ A leírtak nem azt jelentik, hogy az ütközés miatt ki kell dobunk az iménti $h(x) = x\%10$ módosító függvényünket, ugyanis az ütközés feloldására létezik megoldás.
- ◊ Ütközésfeloldásra a következő megoldásokat használjuk:

◊– Láncolás (chaining)

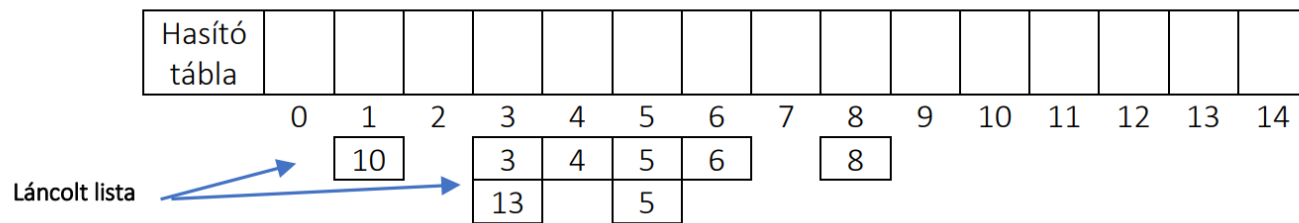
- ◊ Az adatrekordokat m darab S1L listában tároljuk. A hasítótábla $T[i]$ eleme az i . láncolt lista első elemére mutató pointer (vagy $\emptyset$).
- ◊ Ez a lista azokat a k kulcsú elemeket tartalmazza, amikre $h(k) = i$. Beszúráskor a megfelelő lista elejére szúrjuk be az új elemet azért, hogy a beszúrás minél egyszerűbb legyen (továbbá, mert a megfigyelések szerint az újabb elemekhez nagyobb valószínűséggel szeretnénk a közeljövőben hozzáférni).
- ◊ A beszúrás előtt ellenőrizzük a megfelelő listát, a duplikált kulcsok elkerülése céljából. $T:E1*[m]$

A zárt hasítás láncolt listával a következőképpen orvosolja az ütközés problémáját:

Many-One
 $h(x) = x\%10$

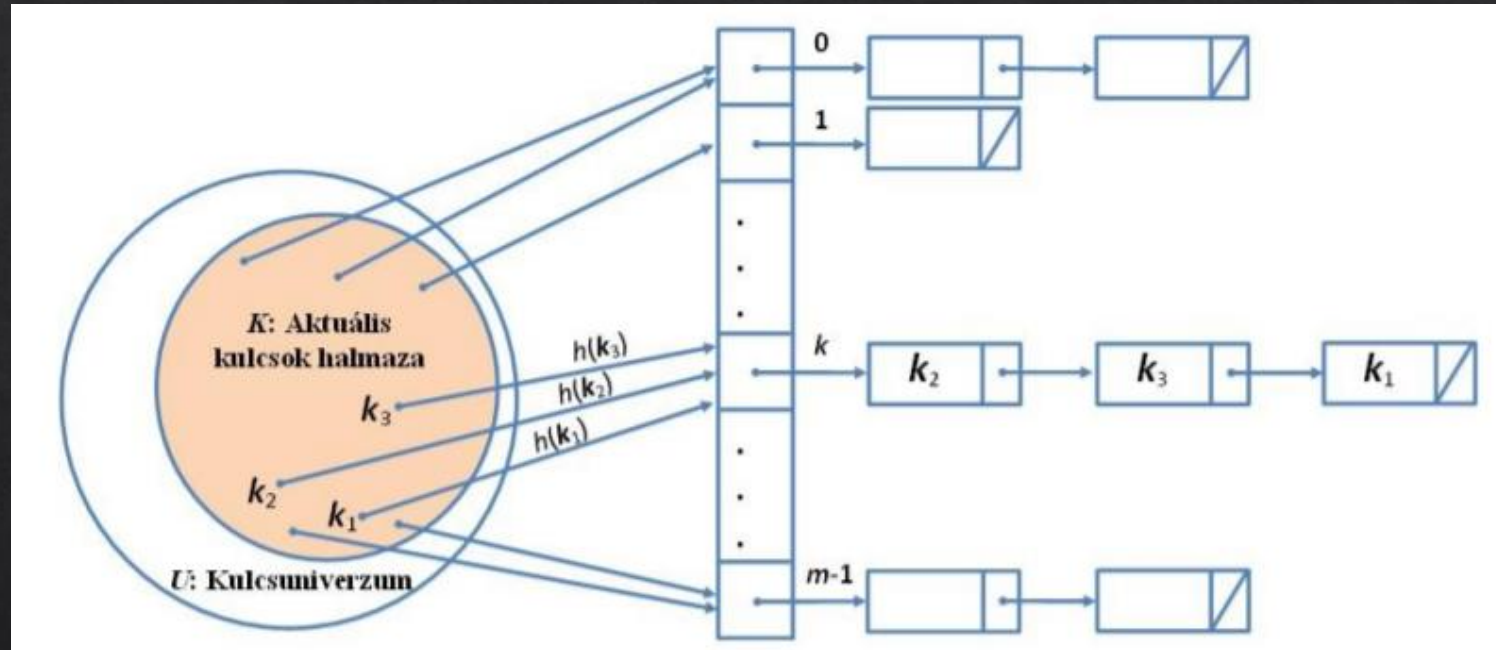
Tárolandó kulcsok

8, 3, 13, 6, 4, 10, 5, 50



Hasítás jelensége, kulcsütközés feloldása láncolással

- ◊ Egy nagyon jó ábra dr Fekete István jegyzetéből:
- ◊ U – kulcsuniverzum $|U| \gg n$
- ◊ $h: U \rightarrow 0..m-1$ hasító függvény
- ◊ $|U| \gg m$
- ◊ A k kulcsú rekordot a hasító tábla $T[h(k)]$ részében (rekeszében) helyezzük el: $T[0..m-1]$ indexű.
- ◊ Ha k_1, k_2 kulcsokra $h(k_1)=h(k_2)$, akkor kulcsütközésről beszélünk.
- ◊ Kulcsütközés feloldása jellemzően két módon történik: láncolt hasító tábla, nyílt címzéses hasító tábla.
- ◊ Példul, ha k egy nem negatív egész, akkor $h(k) = k \bmod m$ egy lehet egy hasító függvény.



Hasító függvény

- ◊ A $h : U \rightarrow [0..m)$ függvény egyszerű egyenletes hasítás, ha a kulcsokat a rések között egyenletesen szórja szét, azaz hozzávetőleg ugyanannyi kulcsot képez le az m rés mindegyikére. Tetszőleges hasító függvénnyel kapcsolatos elvárás, hogy egyszerű egyenletes hasítás legyen.
- ◊ **Több módszer létezik:**
 - ◊ **1 - Osztó módszer:**
 - ◊ **2 - Szorzó módszer:**

I – 1: Osztó módszer:

- ◊ Hasító függvényekre **több módszer létezik:**
- ◊ **1 - Osztó módszer:**
 - ◊ $h(k) = k \bmod m$
 - ◊ Egyenletessége nagyon függ az m megválasztásától, érdemes prímnek választani. Legyen távol a szomszédos kettő hatványoktól.
 - ◊ Ha például a kulcsütközést láncolással szeretnénk feloldani, és kb. 2000 rekordot szeretnénk tárolni $\alpha = 3$ kitöltöttségi aránnyal, akkor a 701 jó választás A 701 ui. olyan prímszám, ami közel esik a $2000/3$ -hoz, de a szomszédos kettő hatványoktól, az 512-től és az 1024-től is elég távol van.
 - ◊ Tábla kitöltöttségi aránya: $\alpha = n/m$ (n rekordok száma a táblában, m -tábla réseinek száma). Láncolt hasító tábla esetén ez ad becslést a listák hosszára, ha egyenletes a hasítás, a listák várható hossza: α

I – 1: Osztó módszer példa

◆ **Feladat:**

$m = 7$, $h : N \rightarrow 0..6$, $h(k) = k \bmod 7$

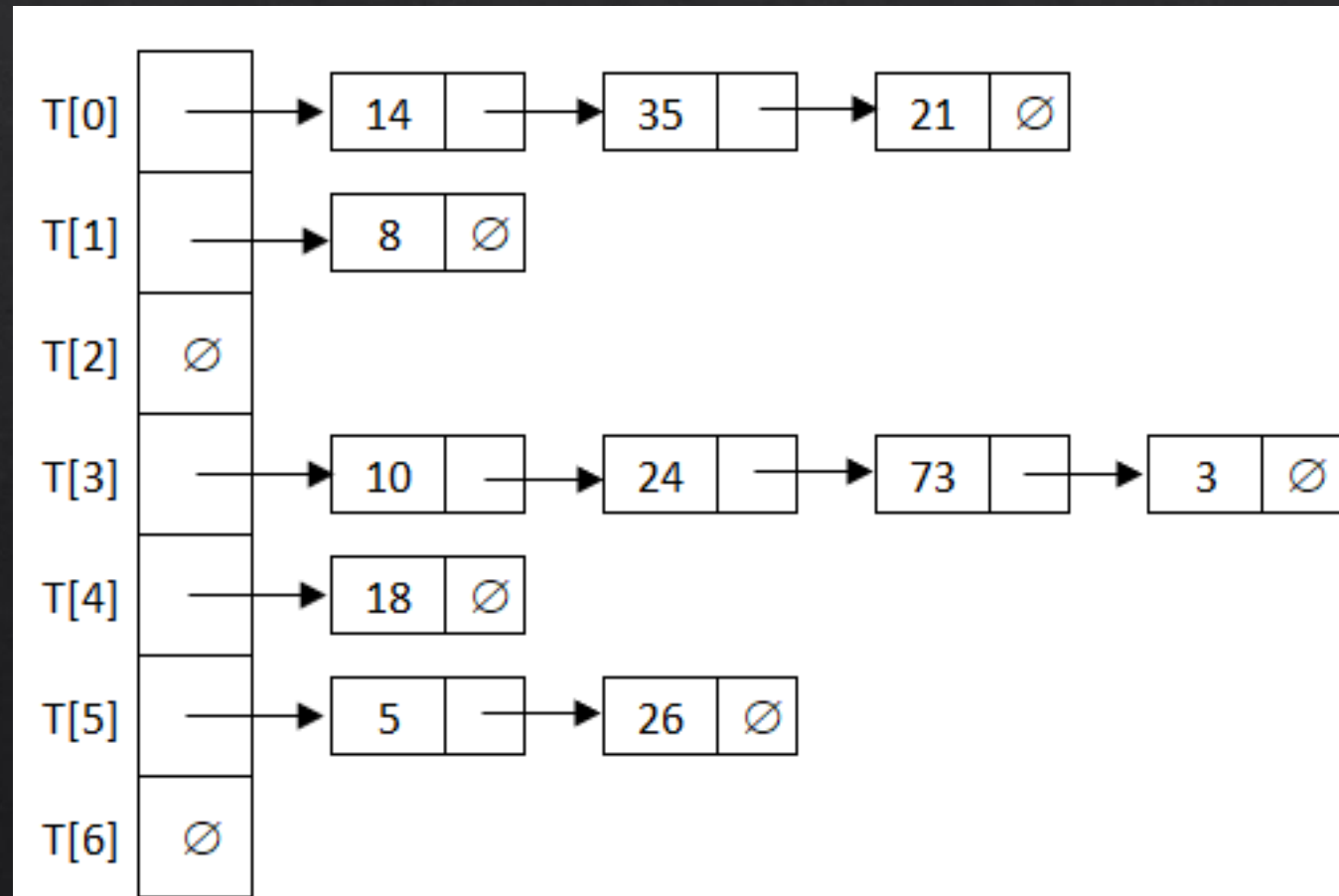
Beszúrandó elemek: 3, 21, 73, 8, 24, 35, 26, 14, 5, 10, 18

◆ k : az elem kulcsa

◆ m : a tábla mérete

◆ $h(k)$: a címhez tartozó
hasítófüggvény

k	h(k)
3	3
21	0
73	3
8	1
24	3
35	0
26	5
14	0
5	5
10	3
18	4



I – 2: - Szorzó módszer:

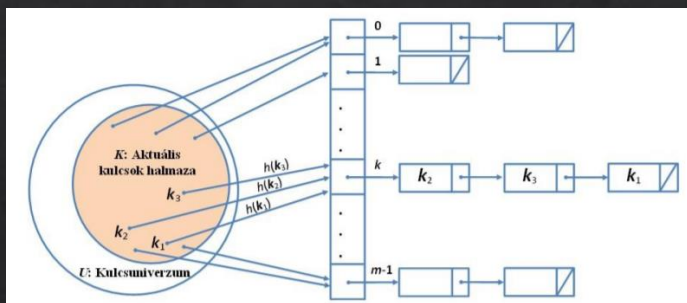
◇ Szorzó módszer:

- ◇ Legyen $0 < A < 1$ egy konstans (ennek megválasztása nehéz feladat).
- ◇ Vegyük a $k \cdot A$ érték tötrészét: $0 \leq \{k \cdot A\} < 1$
- ◇ Beszorzom m -el $\rightarrow h(k) = \lfloor \{k \cdot A\} * m \rfloor \rightarrow h(k) \in [0..m-1]$ azonnal látszik.
- ◇ m megválasztása kevésbé jelentőségteljes az egyenletes hasítás eléréséhez.
- ◇ Érdekesség: D. E. Knuth javaslata az A konstans megválasztására:

$$A \approx \frac{\sqrt{5}-1}{2} \approx 0,6180339887$$

Láncolt hasító tábla algoritmusai a jegyzetből

◇ Rések listái S1L listák



$E1$
$+key : U$
$\dots$ // satellite data may come here
$+next : E1^*$
$+E1() \{ next := \oslash \}$

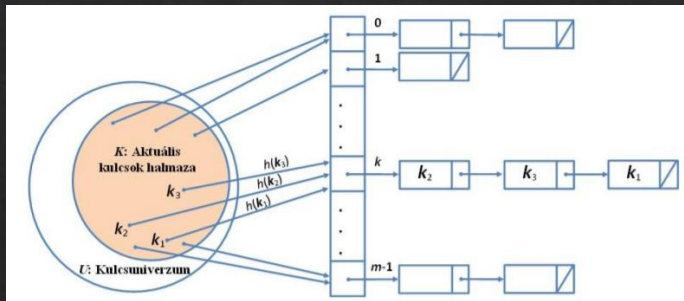
$\text{init}(T:E1^*[m])$
$i := 0 \text{ to } m-1$
$T[i] := \oslash$

$\text{search}(T:E1^*[] ; k:U):E1^*$
return $\text{searchS1L}(T[h(k)],k)$

$\text{searchS1L}(q:E1^* ; k:U):E1^*$
$q \neq \oslash \wedge q \rightarrow key \neq k$
$q := q \rightarrow next$
return q

Láncolt hasító tábla algoritmusai a jegyzetből

◇ Rések listái S1L listák



$E1$
$+key : U$
$\dots$ // satellite data may come here
$+next : E1^*$
$+E1() \{ next := \oslash \}$

$searchS1L(q:E1^* ; k:U):E1^*$
$q \neq \oslash \wedge q \rightarrow key \neq k$
$q := q \rightarrow next$
return q

$insert(T:E1^*[] ; p:E1^*):\mathbb{B}$
$k := p \rightarrow key ; s := h(k)$
$\backslash searchS1L(T[s], k) = \oslash /$
$p \rightarrow next := T[s]$
$T[s] := p$
return $true$
return $false$

$remove(T:E1^*[] ; k:U):E1^*$
$s := h(k) ; p := \oslash ; q := T[s]$
$q \neq \oslash \wedge q \rightarrow key \neq k$
$p := q ; q := q \rightarrow next$
$q \neq \oslash$
$p = \oslash$
$T[s] := q \rightarrow next$
$p \rightarrow next := q \rightarrow next$
$q \rightarrow next := \oslash$
return q
SKIP

Műveletigények

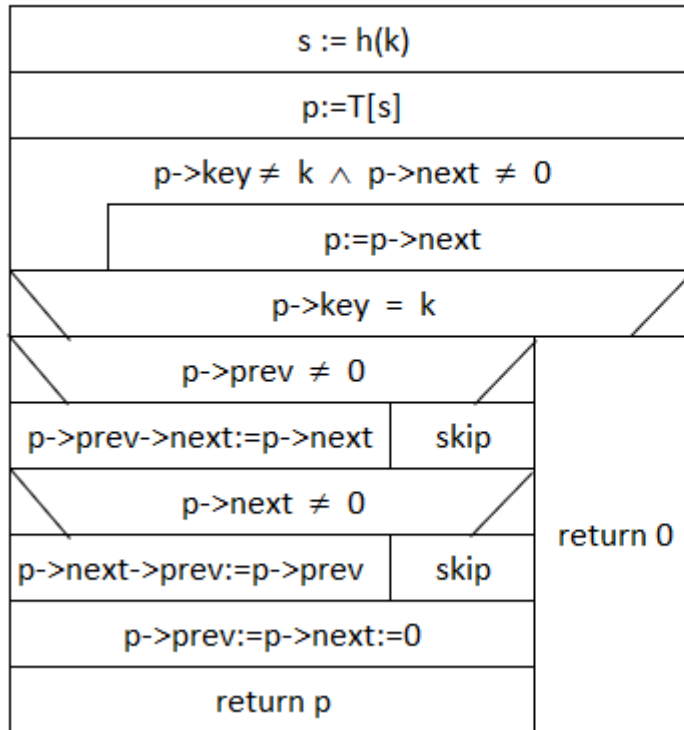
Nyilván $T_{\text{init}}(m) \in \Theta(m)$. A másik három műveletre pedig $mT \in \Theta(1)$, $MT(n) \in \Theta(n)$, $AT(n, m) \in \Theta(1 + \frac{n}{m})$.

$AT(n, m) \in \Theta(1 + \frac{n}{m})$ feltétele, hogy a $h : U \rightarrow 0..(m-1)$ függvény *egyszerű egyenletes hasítás* legyen, és még az indokolja, hogy a résekhez tartozó listák átlagos hossza $= \frac{n}{m} = \alpha$.

Általában feltesszük még, hogy $\frac{n}{m} \in O(1)$. Ebben az esetben nyilván $AT(n, m) \in \Theta(1)$ is teljesül.

Törlés kétirányú lánc esetén

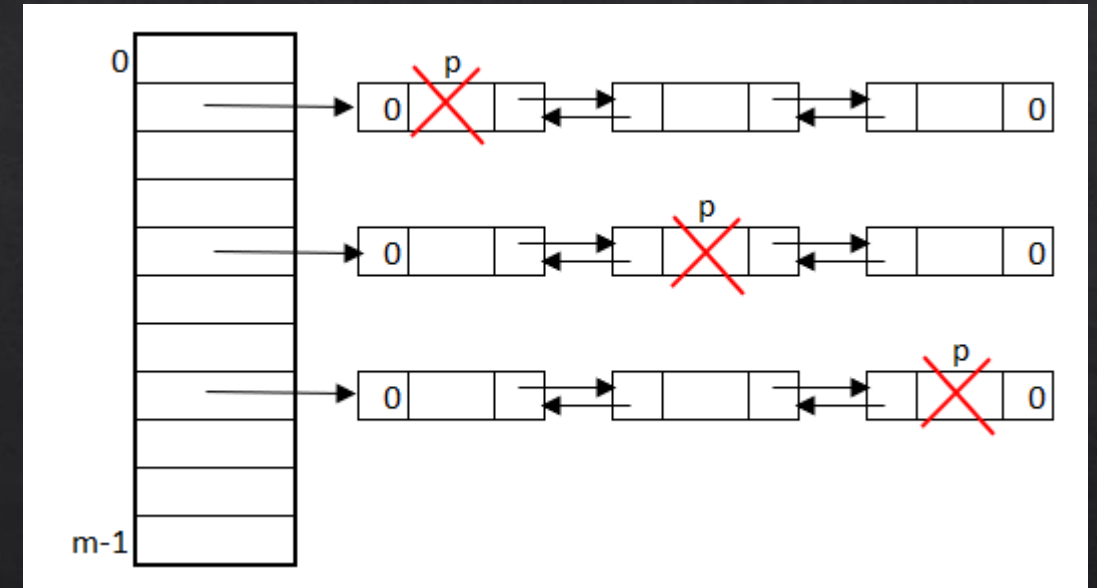
remove($T:E2^*[m]$, $k:U$): $E2^*$



Keresés közben p nem szalad le a listáról.

Így itt biztos, p nem nulla.

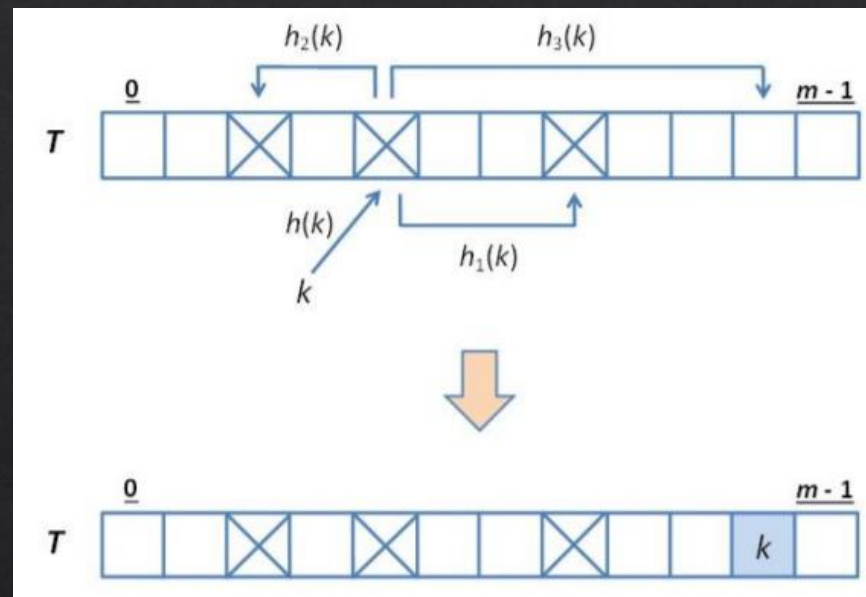
Vigyázni kell, mert nem biztos, hogy p előtt van elem. p után sem mindig van listaelem.



II. Nyílt címzés

Nyíltcímezéses módszerek

- ◊ A rekordok közvetlenül a táblában lesznek elhelyezve.
- ◊ Beszúrásnál, ha az adott rés már foglalt, egy **próbasorozat** mentén megpróbálunk helyet találni az elemnek.
- ◊ A próbasorozat elemei:
- ◊ $h(k,0)$ $h(k,1)$ $h(k,2)$... $h(k,m-1)$
- ◊ Fontos, hogy a próbasorozat bejárja a teljes táblát, ezért a próbasorozat értékei a $0..m-1$ egészek egy permutációja kell legyen.
- ◊ Fontos az is, hogy könnyen reprodukálható legyen. (Adott kulcs esetén mindig ugyanazt a próbasorozatot kell kapjuk.)



Szemléltető ábra dr Fekete István jegyzetéből.

$$h(k)=h(k,0), h_1(k)=h(k,1)$$

$$h_2(k)=h(k,2)$$

Jelölések, fogalmak a nyílt címzéshez

- ◇ $T:R[m]$
- ◇ k egy kulcs, vagy
E –üres a rés D- törölt a rés. E,D extrémális konstansok: $E, D \notin U$
- ◇ $h:U \times 0..(m-1) \rightarrow 0..(m-1)$: hasító próba
 $\langle h(k,0), h(k,1), h(k,2), \dots, h(k,m-1) \rangle$: potenciális próbasorozat
- ◇ Az üres és törölt rések a szabad rések, a többi foglalt rés.
- ◇ Aktuális próbasorozat: a potenciális próbasorozat egy prefixe.
- ◇ Ideális esetben egy tetszőleges potenciális próbasorozat a $\langle 0..m-1 \rangle$ sorozatnak mind az $m!$ permutációját azonos valószínűséggel állítja elő. Ilyenkor egyenletes hasításról beszélünk.
- ◇ Nevezetes próbasorozatok:
 - ◇ **Lineáris, négyzetes és kettős hash** (ez utóbbi tekinthető egyenletesnek)

R
+ $k : U \cup \{E, D\}$ // k is a key or it is Empty or Deleted
+ ... // satellite data

II – 1: Lineáris próbasorozat

Nyílt címzés esetén a műveletek

- ◇ Elsőként nézzük a műveletek algoritmusait arra az esetre, hogy a táblából nem törölünk.
- ◇ **Keresés:** a potenciális próbasorozat szerint haladunk, és keressük a kulcsot. Sikeres keresés esetén megtaláljuk a kulcsot. Ha üres réshez érünk (vagy elvégeztük az összes próbát) leállunk, ekkor sikertelen keresésről beszélünk.
- ◇ **Beszúrás:** a potenciális próbasorozaton haladunk, ha megtaláljuk a kulcsot a beszúrás sikertelen, mert már benne van a táblában a kulcs. Ha üres réshez érünk, már biztosan nem lesz a táblában a kulcs, leállunk és az üres résbe beszúrjuk az elemet. Sikertelen a beszúrás, akkor is, ha nem találtunk üres rekeszt, de elvégeztük az összes próbát, ilyenkor a tábla megtelt.
- ◇ Műveletigények egyenletes hasítást és $0 < \alpha < 1$ kitöltöttségi arányt feltételezve:
- ◇ Sikeres beszúrás és sikertelen keresés legfeljebb: $\frac{1}{1-\alpha}$, 50%-os kitöltöttség esetén ez 2-őt ad.
- ◇ Sikeres keresés és sikertelen beszúrás legfeljebb: $\frac{1}{\alpha} \ln \frac{1}{1-\alpha}$, 50%-os kitöltöttségre ez kb. 1,387
- ◇ $\alpha=0,9$ (90%-os) kitöltöttség esetén az elsőre legfeljebb 10, a másodikra kb. 2,559 adódik.

$h(k)=k \bmod 11$	0	1	2	3	4	5	6	7	8	9	10
		12	1	3	2			7			
lineáris próba:		0	1	2							
		$h(2,0)$	$h(2,1)$	$h(2,2)$							
		2 kulcs sikeres keresése, 2 kulcs sikertelen beszúrása									
					13						
$h(k)=k \bmod 11$	0	1	2	3	4	5	6	7	8	9	10
		12	1	3	2			7			
lineáris próba:		0	1	2	3						
		$h(13,0)$	$h(13,1)$	$h(13,2)$	$h(13,3)$						
		13 kulcs sikertelen keresése, 13 kulcs sikeres beszúrása									

Nyílt címzés esetén a műveletek

- ◊ Hogyan módosulnak a műveletek, ha a táblában a törlést is megengedjük:
- ◊ **Keresés:** a potenciális próbasorozat szerint haladunk, és keressük a kulcsot. D (tartalmú) törölt rekesz esetén folytatódik a művelet, mintha foglalt lenne a rés. Sikeres keresés esetén megtaláljuk a kulcsot. Ha üres réshez érünk (vagy elvégeztük az összes próbát) leállunk, ekkor sikertelen keresésről beszélünk.
- ◊ **Beszúrás:** a potenciális próbasorozaton haladunk, ha megtaláljuk a kulcsot a beszúrás sikertelen, mert már benne van a táblában a kulcs. Ha üres réshez érünk, már biztosan nem lesz a táblában a kulcs, leállunk. A D kulcsú törölt rekeszeket itt is átlépjük, de beszúrásakor az első törölt rekeszt meg kell jegyezni (ha volt), mert oda érdemes majd a kulcsot beszúrni.
- ◊ **Törlés:** kereséssel indul, ha sikeres volt a keresés, a táblából törlődik a kulcs, majd a rés törölt-re állítódik (kulcsa „D” lesz).

$h(k)=k \bmod 11$	0	1	2	3	4	5	6	7	8	9	10
		12	1	D	2			7			
lineáris próba:		0	1	2							
	2 kulcs sikeres keresése, vagy törlése, 2 kulcs sikertelen beszúrása										
$h(k)=k \bmod 11$	0	1	2	3	4	5	6	7	8	9	10
		12	1	D	2			7			
lineáris próba:		0	1	2	3						
	13 kulcs sikertelen keresése vagy törlése, 13 kulcs sikeres beszúrása										
	<i>Ha a próbasorozat mentén volt törölt rés, az első töröltbe kerül majd a 13-as kulcs, hogy a keresés minél kevesebb próbával megtalálja!</i>										

Probléma: nagyszámú törlés következtében elfogynak az üres rekeszek (csak foglalt és törölt lesz), ilyenkor a hatékonyság nagyon elromlik: a táblát újra kell építeni.

Lineáris próbasorozat

- ◇ Lineáris próbasorozat: $h(k,i) = (h(k,0) + i) \bmod m$ $i=0..m-1$

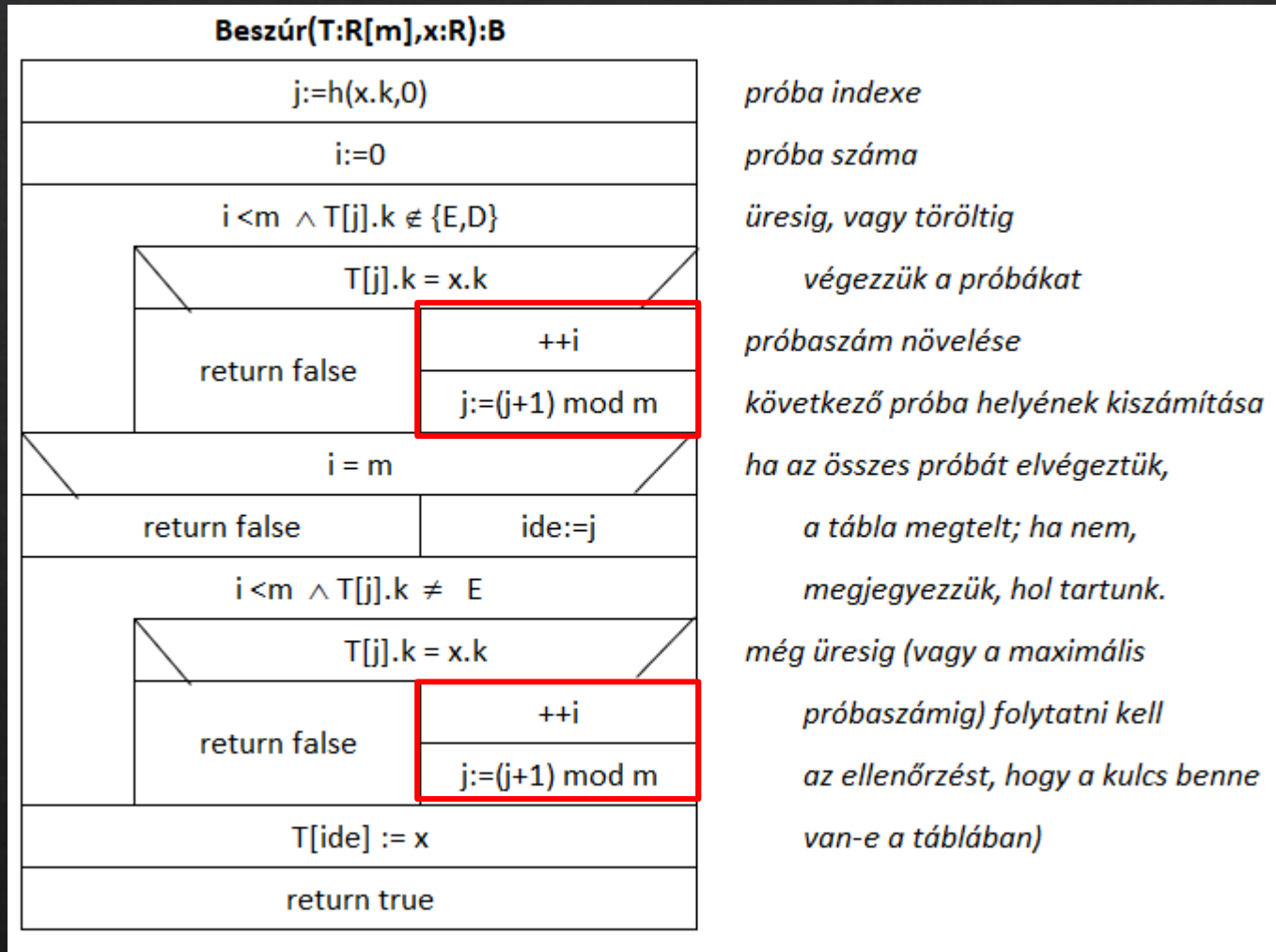
Példa: $m = 11$, $h : \mathbb{N} \rightarrow 0..10$, $h(k,n) = (k + n) \bmod 11$.

Művelet	k	$h_1(k)$	Próbasorozat	0	1	2	3	4	5	6	7	8	9	10
Beszúr	24	2	$2_E \checkmark$			24								
Beszúr	16	5	$5_E \checkmark$			24			16					
Beszúr	57	2	$2, 3_E \checkmark$			24	57		16					
Beszúr	32	10	$10_E \checkmark$			24	57		16					32
Beszúr	15	4	$4_E \checkmark$			24	57	15	16					32
Töröl	57	2	$2, 3_{57} \checkmark$			24	D	15	16					32
Beszúr	21	10	$10, 0_E \checkmark$	21		24	D	15	16					32
Keres	2	2	$2, 3_D, 4, 5, 6_E \times$	21		24	D	15	16					32
Beszúr	2	2	$2, 3_D, 4, 5, 6_E \checkmark$	21		24	2	15	16					32
Beszúr	2	2	$2, 3_2 \times$	21		24	2	15	16					32
Keres	21	10	$10, 0_{21} \checkmark$	21		24	2	15	16					32

$$\alpha = \frac{6}{11}$$

Elsődleges csomósodás jelentkezik, ha a tábla egy szakasza nagyon tele van, pl. 13 beszúrása növeli a 2..5 szakaszban kialakult csomósodás méretét. Ebben a szakaszban megnövekszik az aktuális próbasorozat hossza.

Beszúrás algoritmus lineáris próba esetén



Az algoritmus
könnyen átírható
a másik két
próbasorozatra

II – 2: Négyzetes próba

Négyzetes próba

- ◇ $h(k,i) = (h(k,0) + c_1 * i + c_2 * i^2) \bmod m$ alakú a hasító függvény.
- ◇ c_1 és c_2 konstansokat úgy kell megválasztani, hogy a próbasorozat elérje a tábla valamennyi rekeszét.
- ◇ Ha például m egy 2 hatvány, akkor $c_1=1/2$ és $c_2=1/2$ jó választás.
- ◇ $h(k,i) = (h(k,0) + \frac{1}{2} * i + \frac{1}{2} * i^2) \bmod m$
- ◇ $h(k,i) = (h(k,0) + 1 + 2 + 3 + \dots + i) \bmod m$
- ◇ Szokás „álvéletlen” próbának is nevezni.

Négyzetes próbasorozat

Példa: $m = 8$, $h : \mathbb{N} \rightarrow 0..7$, $h(k, n) = (k + \frac{n+n^2}{2}) \bmod 8$.

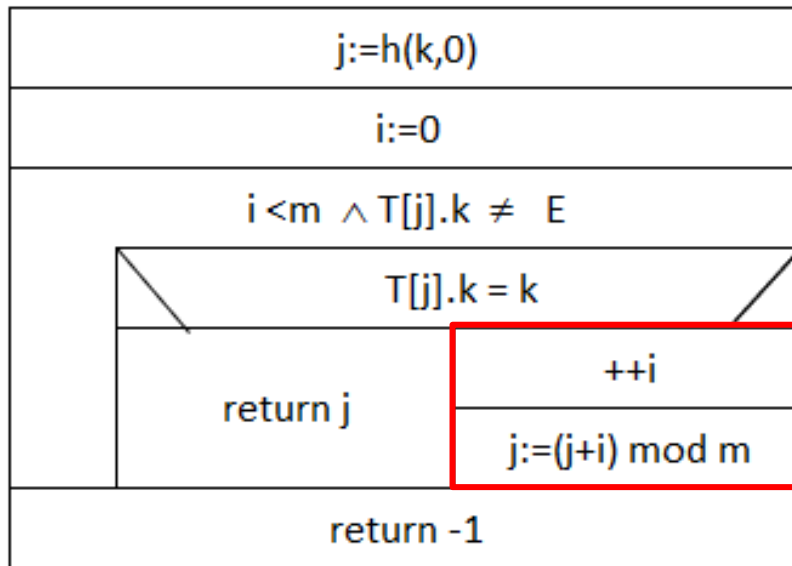
Művelet	k	$h_1(k)$	Próbasorozat	0	1	2	3	4	5	6	7
Beszúr	13	5	$5_E \checkmark$						13		
Beszúr	20	4	$4_E \checkmark$					20	13		
Beszúr	31	7	$7_E \checkmark$					20	13		31
Beszúr	87	7	$7, 0_E \checkmark$	87				20	13		31
Beszúr	12	4	$4, 5, 7, 2_E \checkmark$	87		12		20	13		31
Töröl	31	7	$7_{31} \checkmark$	87		12		20	13		D
Keres	12	4	$4, 5, 7_D, 2_{12} \checkmark$	87		12		20	13		D
Keres	15	7	$7_D, 0, 3_E \times$	87		12		20	13		D
Beszúr	4	4	$4, 5, 7_D, 2, 6_E \checkmark$	87		12		20	13		4
Töröl	10	2	$2, 3_E \times$	87		12		20	13		4
Beszúr	35	3	$3_E \checkmark$	87		12	35	20	13		4
Beszúr	10	2	$2, 3, 5, 0, 4, 1_E \checkmark$	87	10	12	35	20	13		4

$$\alpha = \frac{7}{8}$$

Másodlagos csomósodás jelentkezik, az azonos próbasorozatok mentén. Ezeken a helyeken megnövekszik az aktuális próbasorozat hossza.

Keresés algoritmus négyzetes próba esetén

Keres($T:R[m]$, $k:U$) : Z



próba indexe

próbák száma

Üresig (vagy a maximális próbaszámig)

keressük a kulcsot.

következő próba indexe: mindig eggyel távolabb

ez előző próbától!

Az algoritmus könnyen átírható a másik két próbasorozatra

II – 3: Kettős hasítás

Kettős hasítás

- ◇ Két hasító függvény van, a lineáris próbához hasonló, de nem +1 az eltolás
- ◇ $h_1: U \rightarrow 0..m-1$, megadja a kulcs helyét a táblában
- ◇ $h_2: U \rightarrow 1..m-1$, ez adja meg a lépések hosszát
- ◇ $h(k,i) = (h_1(k) + i \cdot h_2(k)) \bmod m$ alakú a hasító függvény
- ◇ $h_2(k)$ hasító függvénnnyel kapcsolatos elvárások:
 - ◇ $h_2(k)$ legyen nullánál nagyobb, tetszőleges k kulcsra
 - ◇ $h_2(k)$ és m legyenek relatív prímek tetszőleges k kulcsra (m -et prímmnek választják)
- ◇ Jól közelíti az egyenletes hasítást, így kevésbé csomosodik a tábla, hiszen kulcsütközés esetén, azaz ha $h_1(k_1) = h_1(k_2)$, akkor várhatóan $h_2(k_1) \neq h_2(k_2)$ teljesül, és így a kapott potenciális próbasorozat a két elemnél eltérő lesz.

Kettős hasítás próbasorozat

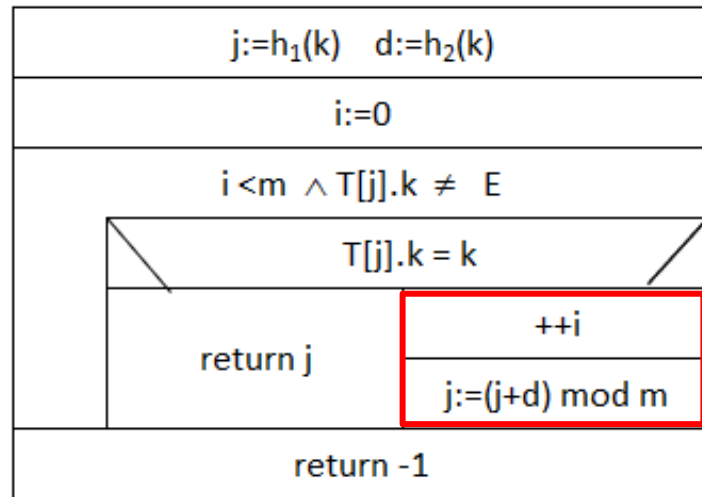
Példa: $m = 11$, $h_1(k) = k \bmod 11$, $h_2(k) = 1 + (k \bmod 10)$.

Művelet	k	$h_1(k)$	$h_2(k)$	Próbasorozat	0	1	2	3	4	5	6	7	8	9	10
Beszúr	23	1	4	$1_E \checkmark$		23									
Beszúr	42	9	3	$9_E \checkmark$		23								42	
Beszúr	31	9	2	$9, 0_E \checkmark$	31	23								42	
Beszúr	110	0	1	$0, 1, 2_E \checkmark$	31	23	110							42	
Beszúr	55	0	6	$0, 6_E \checkmark$	31	23	110				55			42	
Töröl	55	0	6	$0, 6_{55} \checkmark$	31	23	110				D			42	
Keres	72	6	3	$6_D, 9, 1, 4_E \times$	31	23	110				D			42	
Beszúr	14	3	5	$3_E \checkmark$	31	23	110	14			D			42	
Töröl	22	0	3	$0, 3, 6_D, 9, 1, 4_E \times$	31	23	110	14			D			42	
Töröl	110	0	1	$0, 1, 2_{110} \checkmark$	31	23	D	14			D			42	
Beszúr	13	2	4	$2_D, 6_D, 10_E \checkmark$	31	23	13	14			D			42	

$$\alpha = \frac{5}{11}$$

Keresés algoritmus a kettős hasítás esetén

Keres($T:R[m]$, $k:U$) : Z



próba indexe, lépés nagysága

próbák száma

Üresig (vagy a maximális próbaszámig)

keressük a kulcsot.

következő próba indexe: mindig d -vel távolabb

ez előző próbától!